

Gigathon 2026

Python (16-18 lat) III etap

Symulator wyprawy po dwuwymiarowym świecie

Cel:

Stwórz program w Pythonie, który będzie symulatorem wyprawy obiektu po dwuwymiarowym świecie.

Samodzielnie wybierz motyw swojego programu. Może to być na przykład: wyprawa robota, podróż odkrywcy, misja łązika, wędrówka zwierzęcia, rejs statku, ekspedycja w jaskiniach, przejazd kuriera przez miasto albo dowolny inny pomysł.

Program ma prowadzić symulację, zmieniać położenie obiektu, reagować na zdarzenia oraz na końcu wypisać raport z przebiegu wyprawy.

Cały przebieg programu powinien być widoczny w terminalu. Terminal ma pokazywać początek wyprawy, wpisane parametry, kolejne kroki, zdarzenia, zmiany położenia i zasobów oraz koniec wyprawy.

Dodatkowo program powinien pokazać prostą wizualizację trasy w oknie turtle. Turtle służy tylko do narysowania drogi, granic lub ważnych punktów. Nie trzeba robić pełnej gry graficznej.

Najważniejsze jest to, aby program spełniał wymagania opisane poniżej. To jest minimum. Dodatkowe, dobrze działające elementy mogą pomóc zdobyć więcej punktów.

Opis programu

Program powinien działać jako aplikacja uruchamiana z pliku main.py.

Program powinien co najmniej:

- przyjąć od użytkownika początkowe parametry wyprawy,
- używać współrzędnych x oraz y do określania położenia obiektu,
- używać kąta startowego lub kierunku wyrażonego jako kąt,
- obsługiwać zasoby, na przykład: energię, paliwo, zapasy, zdrowie, czas albo wytrzymałość,
- prowadzić symulację w kolejnych krokach, turach albo etapach,
- dodawać elementy świata i losowe zdarzenia wpływające na przebieg wyprawy,
- wypisywać przebieg wyprawy w terminalu krok po kroku,
- narysować prostą trasę wyprawy w turtle,
- zakończyć symulację po spełnieniu określonych warunków i wypisać raport.

Program powinien być zrozumiały dla osoby, która uruchamia go po raz pierwszy. Komunikaty w konsoli powinny jasno informować, co się dzieje i jakie dane należy podać.

Parametry początkowe

Na początku działania program powinien poprosić użytkownika o podanie podstawowych ustawień wyprawy i pokazać krótką informację, co oznaczają.

Program powinien przyjmować co najmniej:

- nazwę wyprawy, obiektu, bohatera lub pojazdu,
- pozycję startową obiektu, czyli wartości x oraz y ,
- kąt startowy, na przykład wartość od 0 do 359 stopni, albo kierunek zamieniany przez program na kąt,
- początkową wartość zasobu, na przykład: energii, paliwa lub zapasów,
- cel wyprawy, rozmiar świata, poziom trudności albo inną zasadę, która wpływa na koniec symulacji.

Możesz dodać własne dodatkowe parametry, na przykład: rodzaj terenu, typ bohatera, rodzaj misji, prędkość, pogodę albo inne ustawienia pasujące do wybranego motywu.

Jeśli użytkownik wpisze błędną wartość, program powinien poprosić o poprawkę albo bezpiecznie przyjąć wartość domyślną.

Świat symulacji

Świat powinien być dwuwymiarowy i opisany współrzędnymi.

W świecie powinny istnieć granice, na przykład: od -100 do 100 na osi x i y albo plansza o ustalonej szerokości i wysokości. Program powinien obsługiwać sytuację, w której obiekt próbuje wyjść poza dozwolony obszar.

W świecie powinny występować co najmniej 3 różne typy elementów wpływających na wyprawę. Mogą to być przeszkody, bonusy, niebezpieczne pola, miejsca odpoczynku, skróty, burze, uszkodzenia, znaleziska, przeciwnicy, punkty kontrolne albo inne elementy pasujące do wybranego motywu.

Element świata powinien coś zmieniać, na przykład: pozycję, zasób, wynik, cel, liczbę kroków albo dalsze decyzje użytkownika.

Tryb tekstowy i przebieg symulacji

Symulacja powinna przebiegać w kolejnych krokach, turach lub etapach. Każdy krok powinien być widoczny w terminalu.

Ruch może być sterowany przez użytkownika albo działać częściowo automatycznie. Możesz samodzielnie wybrać, czy użytkownik wpisuje kierunek, wybiera akcję, ustala strategię, czy program wykonuje ruch według własnych zasad.

W terminalu, przed rozpoczęciem wyprawy, powinny być widoczne co najmniej: nazwa wyprawy, pozycja startowa, kąt startowy, początkowy zasób, granice świata i warunek zakończenia.

W każdym kroku terminal powinien pokazywać co najmniej:

- numer kroku,
- wybraną akcję, ruch albo decyzję programu,
- pozycję przed ruchem i po ruchu,
- stan zasobu przed krokiem i po kroku,
- element świata lub zdarzenie losowe, jeśli wystąpiło,
- krótkie wyjaśnienie, dlaczego zmienił się wynik, pozycja albo zasób.

Nie umieszczaj ważnych informacji tylko w turtle. Osoba oceniająca powinna zrozumieć przebieg wyprawy z samego terminalu.

Wizualizacja w turtle

Okno turtle powinno pokazać prostą graficzną trasę wyprawy. Wystarczy prosta wizualizacja, a nie pełny interfejs gry.

W turtle powinny być widoczne co najmniej: punkt startowy, narysowana trasa, punkt końcowy oraz granice świata albo osie pomagające zrozumieć ruch.

Możesz dodać oznaczenia przeszkód, bonusów, celu lub zdarzeń, ale nie jest to wymagane. Ważniejszy jest poprawny opis kroków w terminalu.

Zdarzenia losowe

Program powinien zawierać co najmniej 2 różne losowe zdarzenia pasujące do motywu programu.

Zdarzenia powinny wpływać na przebieg wyprawy, a nie być wyłącznie ozdobnym komunikatem. Mogą zmieniać zasób, pozycję, wynik, cel, liczbę kroków albo dostępne akcje.

Program powinien jasno wypisać w terminalu, jakie zdarzenie wystąpiło i jaki miało skutek.

Zakończenie programu

Symulacja powinna zakończyć się po spełnieniu co najmniej jednego warunku końcowego. W kodzie powinny istnieć co najmniej 2 możliwe warunki zakończenia, na przykład: dotarcie do celu, brak zasobu, przekroczenie limitu kroków albo wyjście poza świat.

Po zakończeniu program powinien obliczyć wynik wyprawy według samodzielnie ustalonych zasad.

Program ma jasno poinformować użytkownika, czy wyprawa zakończyła się sukcesem, porażką, częściowym sukcesem lub innym możliwym wynikiem, a na końcu wyświetlić raport podsumowujący.

Raport końcowy powinien zawierać co najmniej:

- nazwę wyprawy lub obiektu,
- początkowe parametry wyprawy,
- końcową pozycję,
- liczbę wykonanych kroków,
- pozostały zasób,
- najważniejsze zdarzenia lub odwiedzone elementy świata,

- przyczynę zakończenia wyprawy,
- końcowy wynik.

Raport powinien być czytelny, uporządkowany i wyświetlony w konsoli, niezależnie od okna turtle.

Po zakończeniu wyprawy program powinien pozwolić użytkownikowi uruchomić symulację ponownie z możliwością wpisania nowych parametrów.

Multimedia

Pliki graficzne i dźwiękowe nie są brane pod uwagę podczas oceny. Prosta wizualizacja wykonana w turtle jest oceniana tylko jako pokazanie trasy wyprawy.

Wymagania

1. Zasady

1. Projekt oraz jego fragmenty nie mogą być generowane przez AI.
2. Program musi być kompatybilny z wersją Pythona 3.11.
3. Oddajesz tylko kod programu (.py) oraz plik README.md z krótką instrukcją uruchomienia.
4. Możesz opcjonalnie dodać pliki konfiguracyjne (.json, .toml).
5. Projekt może mieć maksymalnie 10 plików, a ich łączny rozmiar może wynosić maksymalnie 200 kB.
6. Program nie może się łączyć z Internetem.
7. Program ma działać po uruchomieniu main.py bez instalowania dodatkowych bibliotek. Możesz korzystać z modułów ze standardowej biblioteki Pythona (np. math, random, json, os, turtle). Pełna lista dozwolonych modułów: <https://docs.python.org/3.11/library/index.html>

2. Jak udostępnić projekt do oceny

Projekt udostępniamy, wysyłając link do publicznego repozytorium na GitHub.

1. Wejdź na GitHub (<https://www.github.com/>). **Zaloguj się** lub kliknij „**Sign up**”. Jeśli tworzysz nowe konto, potwierdź e-mail.
2. Kliknij + → **New repository**.
3. Wpisz nazwę repozytorium i w sekcji „**Choose visibility**” wybierz „**Public**”.
4. Kliknij **Create repository**.
5. Dodaj pliki do repozytorium w dowolny sposób. Najprostsza metoda: kliknij „**uploading an existing file**”, dodaj pliki i wybierz „**Commit changes**”.
6. Skopiuj link do repozytorium z paska adresu przeglądarki.
7. Dla pewności możesz otworzyć link w trybie prywatnym/Incognito i sprawdzić, czy projekt jest wciąż widoczny.
8. Wklej link do panelu zgłoszeniowego.